



Visit <http://www.fenicsproject.org/> for the latest version of this manual.
Send comments and suggestions to dolfin-dev@fenics.org.

Contents

1	Linear algebra	7
1.1	Matrices and vectors	7
1.1.1	Sparse matrices	8
1.1.2	Dense matrices	8
1.1.3	The common interface	9
1.2	Solving linear systems	9
1.2.1	Iterative methods	9
1.2.2	Multilevel methods	10
1.2.3	Direct methods	10
1.3	Linear algebra backends	11
1.3.1	PETSc	11
1.3.2	JANPACK	11
2	The mesh	13
2.1	Basic concepts	13
2.1.1	Mesh	13
2.1.2	Mesh entities	13
2.2	Mesh iterators	14
2.3	Mesh functions	14
2.4	Mesh refinement	14
2.5	Working with meshes	15
2.5.1	Reading a mesh from file	15
2.5.2	Extracting a boundary mesh	15

2.5.3	Built-in meshes	16
2.5.4	Creating meshes	16
3	Functions	17
3.1	Basic properties	17
3.1.1	Representation	17
3.1.2	Assignment	17
3.1.3	Output	18
3.2	Discrete functions	18
3.3	User-defined functions	18
3.3.1	Cell-dependent functions	19
4	Input/output	21
4.1	Files and objects	21
4.2	File formats	22
4.2.1	DOLFIN XML	22
4.2.2	DOLFIN Binary	23
4.2.3	VTK	23
4.2.4	GNU Octave	24
4.2.5	MATLAB	24
4.3	Converting between file formats	24
4.4	A note on new file formats	25
5	The log system	27
5.1	Generating log messages	27
5.2	Warnings and errors	28
5.3	Debug messages and assertions	28
5.4	Task notification	29
5.5	Progress bars	29
5.6	Controlling the destination of output	30
6	Parameters	31

6.1	Retrieving the value of a parameter	31
6.2	Modifying the value of a parameter	32
6.3	Adding a new parameter	32
6.4	Saving parameters to file	33
6.5	Loading parameters from file	33
A	Installation	37
A.1	Installing from source	37
A.1.1	Dependencies and requirements	37
A.1.2	Downloading the source code	39
A.1.3	Compiling the source code	40
A.1.4	Compiling the demo programs	41
A.1.5	Compiling a program against DOLFIN	41
A.2	Debian package	42
A.3	Installing from source under Windows	42
B	DOLFIN Binary file format	43
B.1	General	43
B.1.1	Mesh data	43
B.1.2	Function data	43
B.1.3	Mesh function	44

Chapter 1

Linear algebra

DOLFIN provides a high-performance linear algebra library, including matrices and vectors, a set of linear solvers, and preconditioners. The core part of the functionality is provided through wrappers that provide a common interface to the functionality of the linear algebra libraries PETSc [7] and JANPACK [10].

1.1 Matrices and vectors

The two basic linear algebra data structures are the classes **Matrix** and **Vector**, representing a sparse $M \times N$ matrix and a vector of length N respectively.

The following code demonstrates how to create a matrix and a vector:

```
Matrix A(M, N);  
Vector x(N);
```

Alternatively, the matrix and the vector may be created by

```
Matrix A;  
Vector x;  
  
A.init(M, N);  
x.init(N);
```

The following code demonstrates how to access the size and the elements of a matrix and a vector:

```
A(5, 5) = 1.0;  
real a = A(4, 3);  
  
x(3) = 2.0;  
real b = x(5);
```

```
unsigned int M = A.size(0);
unsigned int N = A.size(1);

N = x.size();
```

In addition, **DOLFIN** provides optimized functions for setting the values of a set of entries in a matrix or vector:

```
real block[] = {2, 4, 6};
int rows[] = {0, 1, 2};
int cols[] = {0, 1, 2};

A.set(block, rows, cols, 3);
```

Alternatively, the set of values given by the array `block` can be added to the entries given by the arrays `rows` and `cols`:

```
real block[] = {2, 4, 6};
int rows[] = {0, 1, 2};
int cols[] = {0, 1, 2};

A.add(block, rows, cols, 3);
```

These functions are particularly useful for efficient assembly of a (sparse) matrix from a bilinear form.

1.1.1 Sparse matrices

The default **DOLFIN** class `Matrix` is a sparse matrix, which efficiently represents the discretization of a partial differential equation where most entries in the matrix are zero. Alternatively, the class `SparseMatrix` may be used which is identical with the class `Matrix`¹.

If **DOLFIN** has been compiled with support for PETSc, then the sparse matrix is represented as a sparse PETSc matrix². Alternatively, the class `PETScMatrix` may be used, together with the corresponding class `PETScVector`.

If **DOLFIN** has been compiled without support for PETSc, then the sparse matrix is represented as a uBLAS sparse matrix. Alternatively, the class `uBlasSparseMatrix` may be used, together with the corresponding class `uBlasVector`.

1.1.2 Dense matrices

DOLFIN provides the class `DenseMatrix` for representation of dense matrices. A dense matrix representation is often preferable when computing with matrices of small to moderate size. In particular, accessing individual elements is more efficient with a dense matrix representation.

¹The class `Matrix` is a `typedef` for the class `SparseMatrix`.

²By default, the sparse matrix is represented as a PETSc `MATSEQAIJ` matrix, but other PETSc representations are also available.

A `DenseMatrix` is represented as uBLAS dense matrix and alternatively the class `uBlasDenseMatrix` may be used, together with the corresponding class `uBlasVector`.

1.1.3 The common interface

Although **DOLFIN** differentiates between sparse and dense data structures, the two classes `GenericMatrix` and `GenericVector` define a common interface to all matrices and vectors. Refer to the *DOLFIN programmer's reference* for the exact specification of these interfaces.

The following points should be noted:

1. If you want a specific backend like PETSc, then use `PETScVector/PETScMatrix`.
2. If you don't care about the backend, then use `Vector/Matrix`.
3. If you write a *function* that should be able to accept vectors and matrices from any backend as input, then use `GenericVector/GenericMatrix`.

1.2 Solving linear systems

DOLFIN provides a set of efficient solvers for linear systems of the form

$$Ax = b, \tag{1.1}$$

where A is an $N \times N$ matrix and where x and b are vectors of length N . Both iterative (Krylov subspace) solvers and direct (LU) solvers are provided.

1.2.1 Iterative methods

A linear system may be solved by the GMRES Krylov method as follows:

```
Matrix A;
Vector x, b;

GMRES::solve(A, x, b);
```

Alternatively, the linear system may be solved by first creating an object of the class `KrylovSolver`, which is more efficient for repeated solution of linear systems and also allows the specification of both the Krylov method and the preconditioner:

```
KrylovSolver solver(gmres, ilu);
solver.solve(A, x, b);
```

For uBLAS matrices and vectors, the class `uBlasKrylovSolver` may be used and for PETSc matrices and vectors, the class `PETScKrylovSolver` may be used.

Krylov methods

DOLFIN provides the following set of Krylov methods:

<code>cg</code>	The conjugate gradient method
<code>gmres</code>	The GMRES method
<code>bicgstab</code>	The stabilized biconjugate gradient squared method
<code>default_method</code>	Default choice of Krylov method

Preconditioners

DOLFIN provides the following set of preconditioners:

<code>none</code>	No preconditioning
<code>jacobi</code>	Simple Jacobi preconditioning
<code>sor</code>	SOR, successive over-relaxation
<code>ilu</code>	Incomplete LU factorization
<code>dilu</code>	diagonal Incomplete LU factorization
<code>icc</code>	Incomplete Cholesky factorization
<code>amg</code>	Algebraic multigrid
<code>default_pc</code>	Default choice of preconditioner

1.2.2 Multilevel methods

A linear system may be solved by Algebraic multigrid as follows:

1.2.3 Direct methods

A linear system may be solved by a direct LU factorization as follows:

```
Matrix A;
Vector x, b;

LU::solve(A, x, b);
```

Alternatively, the linear system may be solved by first creating an object of the class `LUSolver`, which may be more efficient for repeated solution of linear systems:

```
LUSolver solver;
solver.solve(A, x, b);
```

For uBLAS matrices and vectors, the class `uBlasLUSolver` may be used and for PETSc matrices and vectors, the class `PETScLUSolver` may be used.

1.3 Linear algebra backends

1.3.1 PETSc

PETSc is a suite of data structures and routines for the scalable (parallel) solution of scientific applications modeled by partial differential equations. It employs the MPI standard for all message-passing communication.

DOLFIN wrappers for PETSc linear algebra is provided through the classes `PETScMatrix` and `PETScVector`. Direct access to the PETSc data structures is available through the member functions `mat()` and `vec()`, which return the PETSc `Mat` and `Vec` pointers respectively. For advanced usage not covered by the common **DOLFIN** interface specified by the classes `GenericMatrix` and `GenericVector`, refer directly to the documentation of PETSc.

1.3.2 JANPACK

JANPACK is a suite of data structures, Krylov and Multilevel (AMG) solvers for solving large linear systems. The library uses an alternative sparse matrix representation, optimized for inserting or modifying entries at a low cost.

DOLFIN wrappers for JANPACK is provided through the classes `JANPACKMatrix` and `JANPACKVector`. Direct access to the data structures is available through the member functions `mat()` and `vec()`, which return a `jp_mat_t` and `jp_vec_t` pointers respectively. For advanced usage not covered by the common **DOLFIN** interface specified by the classes `GenericMatrix` and `GenericVector`, refer directly to the documentation for JANPACK.

Chapter 2

The mesh

2.1 Basic concepts

2.1.1 Mesh

A *mesh* consists of *mesh topology*, *mesh geometry* and *mesh overlap* (when running in parallel). These concepts are implemented by the classes `Mesh`, `MeshTopology`, `MeshGeometry` and `MeshDistributedData`.

2.1.2 Mesh entities

A *mesh entity* is a pair (d, i) , where d is the topological dimension of the mesh entity and i is a unique index of the mesh entity. Mesh entities are numbered within each topological dimension from 0 to $n_d - 1$, where n_d is the number of mesh entities of topological dimension d .

For convenience, mesh entities of topological dimension 0 are referred to as *vertices*, entities of dimension 1 *edges*, entities of dimension 2 *faces*, entities of *codimension* 1 *facets* and entities of codimension 0 *cells*. These concepts are summarized in Table 2.1.

Entity	Dimension	Codimension
Vertex	0	–
Edge	1	–
Face	2	–
Facet	–	1
Cell	–	0

Table 2.1: Named mesh entities.

These concepts are implemented by the classes `MeshEntity`, `Vertex`, `Edge`, `Face`, `Facet`, `Cell`.

2.2 Mesh iterators

Algorithms operating on a mesh can often be expressed in terms of *iterators*. The mesh library provides the general iterator `MeshEntityIterator` for iteration over mesh entities, as well as the specialized mesh iterators `VertexIterator`, `EdgeIterator`, `FaceIterator`, `FacetIterator` and `CellIterator`.

The following code illustrates how to iterate over all incident (connected) vertices of all vertices of all cells of a given mesh:

```
for (CellIterator c(mesh); !c.end(); ++c)
  for (VertexIterator v0(*c); !v0.end(); ++v0)
    for (VertexIterator v1(*v0); !v1.end(); ++v1)
      cout << *v1 << endl;
```

This may alternatively be implemented using the general iterator `MeshEntityIterator` as follows:

```
unsigned int dim = mesh.topology().dim();
for (MeshEntityIterator c(mesh, dim); !c.end(); ++c)
  for (MeshEntityIterator v0(*c, 0); !v0.end(); ++v0)
    for (MeshEntityIterator v1(*v0, 0); !v1.end(); ++v1)
      cout << *v1 << endl;
```

2.3 Mesh functions

A `MeshFunction` represents a discrete function that takes a value on each mesh entity of a given topological dimension. A `MeshFunction` may for example be used to store a global numbering scheme for the entities of a (parallel) mesh, marking sub domains or boolean markers for mesh refinement.

2.4 Mesh refinement

A mesh may be refined uniformly as follows:

```
mesh.refine();
```

A mesh may also be refined locally by supplying a `MeshFunction` with boolean markers for the cells that should be refined as follows:

```
mesh.refine(cell_marker);
```

2.5 Working with meshes

2.5.1 Reading a mesh from file

A mesh may be loaded from a file, either by specifying the file name to the constructor of the class `Mesh`:

```
Mesh mesh("mesh.xml");
```

or by creating a `File` object and streaming to a `Mesh`:

```
File file("mesh.xml");  
Mesh mesh;  
file >> mesh;
```

A mesh may be stored to file as follows:

```
File file("mesh.xml");  
Mesh mesh;  
file << mesh;
```

The **DOLFIN** mesh XML format has changed in **DOLFIN** version 0.6.3. Meshes in the old XML format may be converted to the new XML format using the script `dolfin-convert` included in the distribution of **DOLFIN**. For instructions, type `dolfin-convert --help`.

2.5.2 Extracting a boundary mesh

For any given mesh, a mesh of the boundary of the mesh (if any) may be created as follows:

```
BoundaryMesh boundary(mesh);
```

A `BoundaryMesh` is itself a `Mesh` of the same geometrical dimension and has the topological dimension of the mesh minus one.

The computation of a boundary mesh may also provide mappings from the vertices of the boundary mesh to the corresponding vertices in the original mesh, and from the cells of the boundary mesh to the corresponding facets of the original mesh:

```
MeshFunction<unsigned int> vertex_map,  
MeshFunction<unsigned int> cell_map;  
BoundaryMesh boundary(mesh, vertex_map, cell_map);
```

2.5.3 Built-in meshes

DOLFIN provides functionality for creating simple meshes, such as the mesh of the unit square and the unit cube. The following code demonstrates how to create a 16×16 triangular mesh of the unit square (consisting of $2 \times 16 \times 16 = 512$ triangles) and a $16 \times 16 \times 16$ tetrahedral mesh of the unit cube (consisting of $6 \times 16 \times 16 \times 16 = 24576$ tetrahedra):

```
UnitInterval mesh1D(16);
UnitSquare mesh2D(16, 16);
UnitCube mesh3D(16, 16, 16);
```

► *Developer's note:* We could easily add other built-in meshes, like the unit disc, the unit sphere, rectangles, blocks etc. Any contributions are welcome.

2.5.4 Creating meshes

Simplicial meshes (meshes consisting of intervals, triangles or tetrahedra) may be constructed explicitly by specifying the cells and vertices of the mesh. A specialized interface for creating simplicial meshes is provided by the class **MeshEditor**. The following code demonstrates how to create a very simple mesh consisting of two triangles covering the unit square:

```
Mesh mesh;
MeshEditor editor(mesh, CellType::triangle, 2, 2);
editor.initVertices(4);
editor.initCells(2);
editor.addVertex(0, 0.0, 0.0);
editor.addVertex(1, 1.0, 0.0);
editor.addVertex(2, 1.0, 1.0);
editor.addVertex(3, 0.0, 1.0);
editor.addCell(0, 0, 1, 2);
editor.addCell(1, 0, 2, 3);
editor.close();
```

Note that the **DOLFIN** mesh library is not specialized to simplicial meshes, but supports general collections of mesh entities. However, tools like mesh refinement and mesh editors are currently only available for simplicial meshes.

Chapter 3

Functions

► *Developer's note:* Since this chapter was written, the `Function` class has seen a number of improvements which are not covered here. Chapter needs to be updated.

The central concept of a function on a domain $\Omega \subset \mathbb{R}^d$ is modeled by the class `Function`, which is used in **DOLFIN** to represent coefficients or solutions of partial differential equations.

3.1 Basic properties

The following basic properties hold for all `Functions`:

- A `Function` can be scalar or vector-valued;
- A `Function` can be restricted (interpolated) to each local `Cell` of a `Mesh`;
- The underlying representation of a `Function` may vary.

Depending on the actual underlying representation of a `Function`, it may also be possible to evaluate a `Function` at any given `Point`.

3.1.1 Representation

Currently supported representations of `Functions` include *discrete Functions*, *user-defined Functions* and *constant Functions*. These are discussed in detail below.

3.1.2 Assignment

One `Function` may be assigned to another `Function`:

```
Function v;  
Function u = v;
```

3.1.3 Output

A **Function** can be written to a file in various file formats. To write a **Function** `u` to file in VTK format, suitable for viewing in ParaView or MayaVi, create a file with extension `.pvd`:

```
File file("solution.pvd");
file << u;
```

For further details on available file formats, see Chapter 4.

3.2 Discrete functions

A discrete **Function** is defined in terms of a **Vector** of degrees of freedom, a **Mesh**, a local-to-global mapping (**DofMap**) and a finite element. In particular, a discrete **Function** is given by a linear combinations of basis functions:

$$v = \sum_{i=1}^N v_i \phi_i, \quad (3.1)$$

where $\{\phi_i\}_{i=1}^N$ is the global basis of the finite element space defined by the **Mesh** and the finite element, and the nodal values $\{v_i\}_{i=1}^N$ are given by the values of a **Vector**.

3.3 User-defined functions

In the simplest case, a user-defined **Function** is just an expression in terms of the coordinates and is typically used for defining source terms and initial conditions. For example, a source term could be given by

$$f = f(x, y, z) = xy \sin(z/\pi). \quad (3.2)$$

A user-defined **Function** may be defined by creating a sub class of **Function** and overloading the `eval()` function. The following example illustrates how to create a **Function** representing the function in (3.2):

```
class Source : public Function
{
public:

    Source(Mesh& mesh) : Function(mesh) {}

    real eval(const real* x) const
    {
        return x[0]*x[1]*sin(x[2] / DOLFIN_PI);
    }
};

Source f;
```

► *Developer's note:* Write about how to define vector-valued functions.

3.3.1 Cell-dependent functions

In some cases, it may be convenient to define a **Function** in terms of properties of the current **Cell**. One such example is a **Function** that at any given point takes the value of the mesh size at that point.

The following example illustrates how to create such a **Function** by overloading the `eval()` function:

```
class MeshSize : public Function
{
public:

    MeshSize(Mesh& mesh) : Function(mesh) {}

    real eval(const real* x) const
    {
        return cell().diameter();
    }

};

MeshSize h;
```

Note that the current **Cell** is only available during assembly and has no meaning otherwise. For example, it is not possible to write the **Function** `h` to file.

Chapter 4

Input/output

DOLFIN relies on external programs for pre- and post-processing, which means that computational meshes must be imported from file (pre-processing) and computed solutions must be exported to file and then imported into another program for visualization (post-processing). To simplify this process, **DOLFIN** provides support for easy interaction with files and includes output formats for a number of visualization programs.

4.1 Files and objects

A file in **DOLFIN** is represented by the class `File` and reading/writing data is done using the standard C++ operators `>>` (read) and `<<` (write).

Thus, if `file` is a `File` and `object` is an object of some class that can be written to file, then the object can be written to file as follows:

```
file << object;
```

Similarly, if `object` is an object of a class that can be read from file, then data can be read from file (overwriting any previous data held by the object) as follows:

```
file >> object;
```

The format (type) of a file is determined by its filename suffix, if not otherwise specified. Thus, the following code creates a `File` for reading/writing data in **DOLFIN** XML format:

```
File file("data.xml");
```

A complete list of file formats and corresponding file name suffixes is given in Table 4.1.

Alternatively, the format of a file may be explicitly defined. One may thus create a file named `data.xml` for reading/writing data in GNU Octave format:

```
File file("data.xml", File::octave);
```

Suffix	Format	Description
.xml/.xml.gz	File::xml	DOLFIN XML
.bin	File::binary	DOLFIN Binary
.pvd	File::vtk	VTK
.m	File::octave	GNU Octave
(.m)	File::matlab	MATLAB

Table 4.1: File formats and corresponding file name suffixes.

Although many of the classes in **DOLFIN** support file input/output, it is not supported by all classes and the support varies with the choice of file format. A summary of supported classes/formats is given in Table 4.2.

Format	Vector	Matrix	Mesh	Function	Parallel
File::xml	in/out	in/out	in/out	—	Partial
File::binary	in/out	—	in/out	in/out	Yes
File::vtk	—	—	out	out	Yes
File::octave	out	out	out	out	No
File::matlab	out	out	out	out	No

Table 4.2: Matrix of supported combinations of classes and file formats for input/output in **DOLFIN**.

4.2 File formats

In this section, we give some pointers to each of the file formats supported by **DOLFIN**. For detailed information, we refer to the respective user manual of each format/program.

► *Developer's note:* This section needs to be improved and expanded. Any contributions are welcome.

4.2.1 DOLFIN XML

DOLFIN XML is the native format of **DOLFIN**. As the name says, data is stored in XML ASCII format. This has the advantage of being a robust and human-readable format, and if the files are compressed there is little overhead in terms of file size compared to a binary format.

DOLFIN automatically handles gzipped XML files, as illustrated by the following example which reads a **Mesh** from a compressed **DOLFIN** XML file and saves the mesh to an uncompressed **DOLFIN** XML file:

```
Mesh mesh;

File in("mesh.xml.gz");
in >> mesh;
```

```
File out("mesh.xml");  
out << mesh;
```

The same thing can of course be accomplished by

```
# gunzip -c mesh.xml.gz > mesh.xml
```

on the command-line.

4.2.2 DOLFIN Binary

DOLFIN Binary is the new native HPC format of **DOLFIN** and the only format with support for parallel I/O. As the name says, data is stored in uncompressed binary with some additionally header information. A complete specification of the file format can be found in appendix [B](#)

4.2.3 VTK

Data saved in VTK format [\[8\]](#) can be visualized using various packages. The powerful and freely available ParaView [\[6\]](#) is recommended. Alternatively, VTK data can be visualized in MayaVi [\[3\]](#), which is recommended for quality vector PostScript output. Time-dependent data is handled automatically in the VTK format.

The below code illustrates how to export a function in VTK format:

```
Function u;  
  
File out("data.pvd");  
out << u;
```

The sample code produces the file `data.pvd`, which can be read by ParaView. The file `data.pvd` contains a list of files which contain the results computed by **DOLFIN**. For the above example, these files would be named `dataXXX.vtu`, where `XXX` is a counter which is incremented each time the function is saved. If the function `u` was to be saved three times, the files

```
data000000.vtu  
data000001.vtu  
data000002.vtu
```

would be produced. Individual snapshots can be visualized by opening the desired file with the extension `.vtu` or `.pvd` using ParaView.

4.2.4 GNU Octave

GNU Octave [4] is a free clone of MATLAB that can be used to visualize solutions computed in **DOLFIN**, using the commands `pdemesh`, `pdesurf` and `pdeplot`. These commands are normally not part of GNU Octave but are provided by **DOLFIN** in the subdirectory `src/utils/octave/` of the **DOLFIN** source tree. These commands require the external program `ivview` included in the open source distribution of Open Inventor [5]. (Debian users install the package `inventor-clients`.)

To visualize a solution computed with **DOLFIN** and exported in GNU Octave format, first load the solution into GNU Octave by just typing the name of the file without the `.m` suffix. If the solution has been saved to the file `poisson.m`, then just type

```
octave:1> poisson
```

The solution can now be visualized using the command

```
octave:2> pdesurf(points, cells, u)
```

or to visualize just the mesh, type

```
octave:3> pdesurf(points, edges, cells)
```

4.2.5 MATLAB

Since MATLAB [2] is not free, users are encouraged to use GNU Octave whenever possible. That said, data is visualized in much the same way in MATLAB as in GNU Octave, using the MATLAB commands `pdemesh`, `pdesurf` and `pdeplot`.

4.3 Converting between file formats

DOLFIN supplies a script for easy conversion between different file formats. The script is named `dolfin-convert` and can be found in the directory `src/utils/convert/` of the **DOLFIN** source tree. The only supported file formats are currently the Medit `.mesh` format, the Gmsh `.msh` version 2.0 format and the **DOLFIN** XML (`.xml`) mesh format.

To convert a mesh in Medit `.mesh` format generated by TetGen with the `-g` option, type

```
# dolfin-convert mesh.mesh mesh.xml
```

To convert a mesh in Gmsh `.msh` format type

```
# dolfin-convert mesh.msh mesh.xml
```


In generating a Gmsh mesh, make sure to define a physical surface/volume. It is also possible to convert a mesh from the old **DOLFIN** XML (.xml) mesh format to the current one by typing

```
# dolfin-convert -i xml-old old_mesh.xml new_mesh.xml
```

Example meshes can be found in the directory `src/utils/convert/` of the **DOLFIN** source tree.

4.4 A note on new file formats

With some effort, **DOLFIN** can be expanded with new file formats. Any contributions are welcome. If you wish to contribute to **DOLFIN**, then adding a new file format (or improving upon an existing file format) is a good place to start. Take a look at one of the current formats in the subdirectory `src/kernel/io/` of the **DOLFIN** source tree to get a feeling for how to design the file format, or ask at `dolfin-dev@fenics.org` for directions.

Also consider contributing to the `dolfin-convert` script by adding a conversion routine for your favorite format. The script is written in Python and should be easy to extend with new formats.g

Chapter 5

The log system

DOLFIN provides a simple interface for uniform handling of log messages, including warnings and errors. All messages are collected to a single stream, which allows the destination and formatting of the output from an entire program, including the **DOLFIN** library, to be controlled by the user.

5.1 Generating log messages

Log messages can be generated using the function `message()` available in the `dolfin` namespace:

```
void message(std::string format, ...);
```

which works similarly to the standard C library function `printf`. The following examples illustrate the usage of `message()`:

```
message("Solving linear system.");
message("Size of vector: %d.", x.size());
message("R = %.3e (TOL = %.3e)", R, TOL);
```

As an alternative to `message()`, **DOLFIN** provides a C++ style interface to generating log messages. Thus, the above examples can also be implemented as follows:

```
cout << "Solving linear system." << endl;
cout << "Size of vector: " << x.size() << "." << endl;
cout << "R = " << R << " (TOL = " << TOL << ")" << endl;
```

Note the use of `dolfin::cout` and `dolfin::endl` from the `dolfin` namespace, corresponding to the standard `std::cout` and `std::endl` in namespace `std`. If log messages are directed to standard output (see below), then `dolfin::cout` and `std::cout` may be mixed freely.

Most classes provided by **DOLFIN** can be used together with `dolfin::cout` and `dolfin::endl` to display short informative messages about objects:

```
Matrix A(10, 10);  
cout << A << endl;
```

To display detailed information for an object, use the member function `disp()`:

```
Matrix A(10, 10);  
A.disp();
```

Use with caution for large objects. For a `Matrix`, calling `disp()` displays all matrix entries.

5.2 Warnings and errors

Warnings and error messages can be generated using the functions

```
warning(std::string format, ...);  
error(std::string format, ...);
```

Once an error is encountered, the program throws an exception.

5.3 Debug messages and assertions

The macro `dolfin_debug()` works similarly to `message()`:

```
dolfin_debug(message);
```

but in addition to displaying the given message, information is printed about the location of the code that generated the debug message (file, function name and line number).

Note that in order to pass formatting strings and additional arguments with debug messages, the variations `dolfin_debug1()`, `dolfin_debug2()` and so on, depending on the number of arguments, must be used.

Assertions can often be a helpful programming tool. Use assertions whenever you assume something about a variable in your code, such as assuming that given input to a function is valid. **DOLFIN** provides the macro `dolfin_assert()` for creating assertions:

```
dolfin_assert(check);
```

This macro accepts a boolean expression and if the expression evaluates to false, an error message is displayed, including the file, function name and line number of the assertion, and a segmentation fault is raised (to enable easy attachment to a debugger). The following examples illustrate the use of `dolfin_assert()`:

```
dolfin_assert(i >= 0);
dolfin_assert(i < n);
dolfin_assert(cell.type() == Cell::triangle);
dolfin_assert(cell.type() == Cell::tetrahedron);
```

Note that assertions are only active when compiling **DOLFIN** and your program with `DEBUG` defined (configure option `--enable-debug` or compiler flag `-DDEBUG`). Otherwise, the macro `dolfin_assert()` expands to nothing, meaning that liberal use of assertions does not affect performance, since assertions are only present during development and debugging.

5.4 Task notification

The two functions `dolfin_begin()` and `dolfin_end()` available in the `dolfin` name space can be used to notify the **DOLFIN** log system about the beginning and end of a task:

```
void begin();
void end();
```

Alternatively, a string message (or a formatting string with optional arguments) can be supplied:

```
void begin(std::string format, ...);
void end();
```

These functions enable the **DOLFIN** log system to display messages, warnings and errors hierarchically, by automatically indenting the output produced between calls to `begin()` and `end()`. A program may contain an arbitrary number of nested tasks.

5.5 Progress bars

The **DOLFIN** log system provides the class `Progress` for simple creation of progress sessions. A progress session automatically displays the progress of a computation using a progress bar.

If the number of steps of a computation is known, a progress session should be defined in terms of the number of steps and updated in each step of the computation as illustrated by the following example:

```
Progress p("Assembling", mesh.noCells());
for (CellIterator c(mesh); !c.end(); ++c)
{
    ...
    p++;
}
```

It is also possible to specify the step number explicitly by assigning an integer to the progress session:

```
Progress p("Iterating over vector", x.size())
for (uint i = 0; i < x.size(); i++)
{
    ...
    p = i;
}
```

Alternatively, if the number of steps is unknown, the progress session needs to be updated with the current percentage of the progress:

```
Progress p("Time-stepping");
while ( t < T )
{
    ...
    p = t / T;
}
```

The progress bar created by the progress session will only be updated if the progress has changed significantly since the last update (by default at least 10%). The amount of change needed for an update can be controlled using the parameter "progress step":

```
dolfin_set("progress step", 0.01);
```

Note that several progress sessions may be created simultaneously, or nested within tasks.

5.6 Controlling the destination of output

By default, the **DOLFIN** log system directs messages to standard output (the terminal). Messages may also be turned off completely. To specify the destination, set the value of the parameter "output destination":

```
dolfin_set("output destination", "terminal");
dolfin_set("output destination", "silent");
```

One may also set the debug level for the **DOLFIN** log system so that only messages with a debug level higher than or equal to the current debug level are printed:

```
dolfin_set("debug level", "1");
```

Chapter 6

Parameters

► *Developer's note:* Since this chapter was written, the **DOLFIN** parameter system has been completely redesigned and now supports localization of parameters to objects or hierarchies of objects. Chapter needs to be updated.

DOLFIN keeps a global database of parameters that control the behavior of the various components of **DOLFIN**. Parameters are controlled using a uniform type-independent interface that allows retrieving the values of existing parameters, modifying existing parameters and adding new parameters to the database.

6.1 Retrieving the value of a parameter

To retrieve the value of a parameter, use the function `dolfin.get()` available in the `dolfin` namespace:

```
Parameter dolfin_get(std::string key);
```

This function accepts as argument a string `key` and returns the value of the parameter matching the given key. An error message is printed through the log system if there is no parameter with the given key in the database.

The value of the parameter is automatically cast to the correct type when assigning the value of `dolfin.get()` to a variable, as illustrated by the following examples:

```
real TOL = dolfin_get("tolerance");
int num_samples = dolfin_get("number of samples");
bool solve_dual = dolfin_get("solve dual problem");
std::string filename = dolfin_get("file name");
```

Note that there is a small cost associated with accessing the value of a parameter, so if the value of a parameter is to be used multiple times, then it should be retrieved once and stored in a local variable as illustrated by the following example:

```
int num_samples = dolfin_get("number of samples");
for (int i = 0; i < num_samples; i++)
{
    ...
}
```

6.2 Modifying the value of a parameter

To modify the value of a parameter, use the function `dolfin_set()` available in the `dolfin` namespace:

```
void dolfin_set(std::string key, Parameter value);
```

This function accepts as arguments a string `key` together with the corresponding value. The value type should match the type of parameter that is being modified. An error message is printed through the log system if there is no parameter with the given key in the database.

The following examples illustrate the use of `dolfin_set()`:

```
dolfin_set("tolerance", 0.01);
dolfin_set("number of samples", 10);
dolfin_set("solve dual problem", true);
dolfin_set("file name", "solution.xml");
```

Note that changing the values of parameters using `dolfin_set()` does not change the values of already retrieved parameters; it only changes the values of parameters in the database. Thus, the value of a parameter must be changed before using a component that is controlled by the parameter in question.

6.3 Adding a new parameter

To add a parameter to the database, use the function `add()` available in the `dolfin` namespace:

```
void add(std::string key, Parameter value);
```

This function accepts two arguments: a unique key identifying the new parameter and the value of the new parameter.

The following examples illustrate the use of `add()`:

```
add("tolerance", 0.01);
add("number of samples", 10);
add("solve dual problem", true);
add("file name", "solution.xml");
```


6.4 Saving parameters to file

The following code illustrates how to save the current database of parameters to a file in **DOLFIN** XML format:

```
File file("parameters.xml");
file << ParameterSystem::parameters;
```

When running a simulation in **DOLFIN**, saving the parameter database to a file is an easy way to document the set of parameters used in the simulation.

6.5 Loading parameters from file

The following code illustrates how to load a set of parameters into the current database of parameters from a file in **DOLFIN** XML format:

```
File file("parameters.xml");
file >> ParameterSystem::parameters;
```

The following example illustrates how to specify a list of parameters in the **DOLFIN** XML format

```
<?xml version="1.0" encoding="UTF-8"?>

<dolfin xmlns:dolfin="http://www.fenics.org/dolfin/">
  <parameters>
    <parameter name="tolerance" type="real" value="0.01"/>
    <parameter name="number of samples" type="int" value="10"/>
    <parameter name="solve dual problem" type="bool" value="false"/>
    <parameter name="file name" type="string" value="solution.xml"/>
  </parameters>
</dolfin>
```


Bibliography

- [1] *Cygwin*, 2005. <http://cygwin.com/>.
- [2] *MATLAB*, 2005. <http://www.mathworks.com/>.
- [3] *MayaVi*, 2005. <http://mayavi.sourceforge.net/>.
- [4] *Octave*, 2005. <http://www.octave.org/>.
- [5] *Open Inventor*, 2005. <http://http://oss.sgi.com/projects/inventor/>.
- [6] *ParaView*, 2005. <http://www.paraview.org/>.
- [7] *Portable, extensible toolkit for scientific computation petsc*, 2005. <http://www-unix.mcs.anl.gov/petsc/petsc-2/>.
- [8] *The Visualization Toolkit (VTK)*, 2005. <http://www.vtk.org/>.
- [9] *NumPy*, 2006. <http://numpy.scipy.org/>.
- [10] N. JANSSON, *JANPACK*, 2012. <http://www.csc.kth.se/~njansson/janpack>.

Appendix A

Installation

The source code of **DOLFIN** is portable and should compile on any Unix system, although it is developed mainly under GNU/Linux (in particular Debian GNU/Linux). **DOLFIN** can be compiled under Windows through Cygwin [1]. Questions, bug reports and patches concerning the installation should be directed to the **DOLFIN** mailing list at the address

`dolphin-dev@fenics.org`

DOLFIN must currently be compiled directly from source, but an effort is underway to provide precompiled Debian packages of **DOLFIN** and other **FENICS** components.

A.1 Installing from source

A.1.1 Dependencies and requirements

DOLFIN depends on a number of libraries that need to be installed on your system. These libraries include Boost, Libxml2 and optionally PETSc and UMFPACK. If you wish to use the Python interface **PyDOLFIN**, the libraries SWIG and NumPy are required. In addition to these libraries, you need to install **FIAT** and **FFC** if you want to define your own variational forms.

Installing Boost

Boost is a collection of C++ source libraries. Boost can be obtained from

`http://www.boost.org/`

Packages are available for most Linux distributions. For Ubuntu/Debian users, the package to install is `boost-dev`.

Installing Libxml2

Libxml2 is a library used by **DOLFIN** to parse XML data files. Libxml2 can be obtained from

<http://xmlsoft.org/>

Packages are available for most Linux distributions. For Ubuntu/Debian users, the package to install is `libxml2-dev`.

Installing NumPy

NumPy is required for generating the Python interface **PyDOLFIN**. It can be obtained from

<http://numpy.scipy.org/>

Packages are available for most Linux distributions. For Ubuntu/Debian users, the packages `python-numpy` and `python-numpy-ext` should be installed.

Installing SWIG

SWIG is also required for generating the Python interface **PyDOLFIN**. It can be obtained from

<http://www.swig.org/>

Packages are available for most Linux distributions. For Ubuntu/Debian users, the package `swig` should be installed.

Installing UMFPACK

UMFPACK is part of the SuiteSparse collection and is a library for the direct solution of linear systems. It is highly recommended if PETSc is not installed. If PETSc is not installed, UMFPACK is the default direct linear solver. The SuiteSparse collection can be downloaded from

<http://www.cise.ufl.edu/research/sparse/SuiteSparse/>

UMFPACK is available as a package for many Linux distributions. For Ubuntu/Debian users, the packages `libumfpack4` and `libumfpack4-dev` should be installed. For other distributions, the package may be called `libufsparse`.

Installing PETSc

Optionally, **DOLFIN** may be compiled with support for PETSc. To compile **DOLFIN** with PETSc, add the flag `--enable-petsc` during the initial configuration of **DOLFIN**.

PETSc is a library for the solution of linear and nonlinear systems, functioning as the backend for the **DOLFIN** linear algebra classes. **DOLFIN** depends on PETSc version 2.3.1, which can be obtained from

```
http://www-unix.mcs.anl.gov/petsc/petsc-2/
```

Follow the installation instructions on the PETSc web page. Normally, you should only have to perform the following simple steps in the PETSc source directory:

```
# export PETSC_DIR='pwd'
# ./config/configure.py --with-clanguage=cxx --with-shared=1
# make all
```

Add `--download-hypre=yes` to `configure.py` if you want to install Hypre which provides a collection of preconditioners, including algebraic multigrid (AMG), and `--download-umfpack=yes` to `configure.py` if you want to install UMFPACK which provided as fast direct linear solver. Both packages are highly recommended.

DOLFIN assumes that `PETSC_DIR` is `/usr/local/lib/petsc/` but this can be controlled using the flag `--with-petsc-dir=<path>` when configuring DOLFIN (see below).

Installing FFC

DOLFIN uses the FEniCS Form Compiler **FFC** to process variational forms. **FFC** can be obtained from

```
http://www.fenics.org/
```

Follow the installation instructions given in the **FFC** manual. **FFC** follows the standard for Python packages, which means that normally you should only have to perform the following simple step in the **FFC** source directory:

```
# python setup.py install
```

Note that **FFC** depends on **FIAT**, which in turn depends on the Python packages NumPy [9] (Debian package `python-numpy`) and LinearAlgebra (Debian package `python-numpy-ext`). Refer to the **FFC** manual for further details.

A.1.2 Downloading the source code

The latest release of **DOLFIN** can be obtained as a `tar.gz` archive in the download section at

```
http://www.fenics.org/
```

Download the latest release of **DOLFIN**, for example `dolfin-x.y.z.tar.gz`, and unpack using the command

```
# tar xfv dolfin-x.y.z.tar.gz
```

This creates a directory `dolfin-x.y.z` containing the **DOLFIN** source code.

If you want the very latest version of **DOLFIN**, it can be accessed directly from the development repository through `hg` (Mercurial):

```
# hg clone http://www.fenics.org/hg/dolfin
```

This version may contain features not yet present in the latest release, but may also be less stable and even not work at all.

A.1.3 Compiling the source code

DOLFIN is built using the standard GNU Autotools (Automake, Autoconf) and libtool, which means that the installation procedure is simple:

```
# ./configure
# make
```

followed by an optional

```
# make install
```

to install **DOLFIN** on your system.

The configure script will check for a number of libraries and try to figure out how compile **DOLFIN** against these libraries. The configure script accepts a collection of optional arguments that can be used to control the compilation process. A few of these are listed below. Use the command

```
# ./configure --help
```

for a complete list of arguments.

- Use the option `--prefix=<path>` to specify an alternative directory for installation of **DOLFIN**. The default directory is `/usr/local/`, which means that header files will be installed under `/usr/local/include/` and libraries will be installed under `/usr/local/lib/`. This option can be useful if you don't have root access but want to install **DOLFIN** locally on a user account as follows:


```
# mkdir ~/local
# ./configure --prefix=~/local
# make
# make install
```

- Use the option `--enable-debug` to compile **DOLFIN** with debugging symbols and assertions.
- Use the option `--enable-optimization` to compile an optimized version of **DOLFIN** without debugging symbols and assertions.
- Use the option `--disable-curses` to compile **DOLFIN** without the curses interface (a text-mode graphical user interface).
- Use the option `--enable-petsc` to compile **DOLFIN** with support for PETSc.
- Use the option `--disable-pydolfin` to compile without support for PyDOLFIN.
- Use the option `--disable-mpi` to compile **DOLFIN** without support for MPI (Message Passing Interface), assuming PETSc has been compiled without support for MPI.
- Use the option `--with-petsc-dir=<path>` to specify the location of the PETSc directory. By default, **DOLFIN** assumes that PETSc has been installed in `/usr/local/lib/petsc/`.

A.1.4 Compiling the demo programs

After compiling the **DOLFIN** library according to the instructions above, you may want to try one of the demo programs in the subdirectory `src/demo/` of the **DOLFIN** source tree. Just enter the directory containing the demo program you want to compile and type `make`. You may also compile all demo programs at once using the command

```
# make demo
```

A.1.5 Compiling a program against DOLFIN

Whether you are writing your own Makefiles or using an automated build system such as GNU Autotools or BuildSystem, it is straightforward to compile a program against **DOLFIN**. The necessary include and library paths can be obtained through the script `dolfin-config` which is automatically generated during the compilation of **DOLFIN** and installed in the `bin` subdirectory of the `<path>` specified with `--prefix`. Assuming this directory is in your executable path (environment variable `PATH`), the include path for building **DOLFIN** can be obtained from the command

```
dolfin-config --cflags
```

and the path to **DOLFIN** libraries can be obtained from the command

```
dolfin-config --libs
```

If `dolfin-config` is not in your executable path, you need to provide the full path to `dolfin-config`.

Examples of how to write a proper `Makefile` are provided with each of the example programs in the subdirectory `src/demo/` in the **DOLFIN** source tree.

A.2 Debian package

In preparation.

A.3 Installing from source under Windows

DOLFIN can be used under Windows using Cygwin, which provides a Linux-like environment. The installation process is the same as under GNU/Linux. To use **DOLFIN** under Cygwin, the Cygwin development tools must be installed. Instructions for installing PETSc under Cygwin can be found on the PETSc web page. Installation of **FFC** and **FIAT** is the same as under GNU/Linux. The Python package NumPy is not available as a Cygwin package and must be installed manually. To compile **DOLFIN**, the Cygwin package `libxml2-devel` must be installed. For **PyDOLFIN** the package `swig` must be installed. NumPy is not available as a package for Cygwin, therefore it must be installed manually if you wish to use **PyDOLFIN**. The compilation procedure is then the same as under GNU/Linux. If MPI has not been installed:

```
# ./configure --disable-mpi
# make
```

followed by an optional

```
# make install
```

will compile **DOLFIN** on your system.

Appendix B

DOLFIN Binary file format

This chapter gives a detailed description of the **DOLFIN** Binary file format.

B.1 General

Each binary file first contains the following generic header:

```
/* Generic header */
typedef struct {
    uint32_t magic;
    uint32_t bendian;
    uint32_t pe_size;
    uint32_t type;
} header;
```

Where, **magic** is the file's binary magic (0xBABE), **bendian** is endian information, always one if the file were written on a big endian system, **pe_size** contains the number of PEs who wrote the data and **type** is the type of data contained in the file: 0 for Mesh data, 1 for Vector data, 2 for Function(s) data and 3 for Mesh function data.

B.1.1 Mesh data

B.1.2 Function data

If the generic headers type is 2, the rest of the file contains one or several functions. Directly following the generic header the file contains a single integer (**sizeof(int)**) which tells how many functions the rest of the file contains.

```
#define FNAME_LENGTH 256

/* Header for function data */
```

```
typedef struct {  
    uint32_t dim;           /* Geometric dimension of function */  
    uint32_t size;         /* Size of functions data (typically dim * number of vertices) */  
    double t;              /* Simulation time */  
    char name[FNAME_LENGTH]; /* Name of sample (e.g Velocity, Pressure) */  
} fheader;
```

dim: The geometric dimension of the function.

size: Size of the function data (typically dim * number of vertices).

t: Simulation time

name: Name of sample (e.g Velocity, Pressure)

B.1.3 Mesh function

Index

AMGSolver, 10
File, 21
GMRES, 9
GenericMatrix, 9
GenericVector, 9
KrylovSolver, 9
LUSolver, 10
LU, 10
Matrix, 7
Progress, 29
Vector, 7
add(), 32
cout, 27
dolphin_assert(), 28
dolphin_begin(), 29
dolphin_debug(), 28
dolphin_end(), 29
dolphin_get(), 31
dolphin_set(), 32
endl, 27
error(), 28
message(), 27
warning(), 28

algebraic multigrid, 10
AMG, 10
assertions, 28

BiCGStab, 10
Boost, 37

compiling, 40, 41
conjugate gradient method, 10
Cygwin, 42

Debian package, 42
debugging, 28
demo programs, 41
dense matrix, 8
dependencies, 37
direct methods, 10
DOLFIN Binary, 23, 43
downloading, 39
errors, 28

FFC, 39
FIAT, 39
file formats, 22
Function, 17
functions, 17

GMRES method, 9

I/O, 21
ILU, 10
incomplete LU factorization, 10
input/output, 21
installation, 37
iterative methods, 9

Jacobi, 10
JANPACK, 11

Krylov subspace methods, 9

Libxml2, 38
linear algebra backends, 11
linear systems, 9
log system, 27
LU factorization, 10

MayaVi, 18
multilevel methods, 10

NumPy, 38

object, 21
output destination, 30

parameters, 31
ParaView, 18
PETSc, 11, 38
post-processing, 21
pre-processing, 21
preconditioners, 10
progress bar, 29

SOR, 10
source code, 39
sparse matrix, 8
successive over-relaxation, 10
SWIG, 38

tasks, [29](#)

UMFPACK, [38](#)

user-defined functions, [18](#)

warnings, [28](#)

XML, [22](#), [33](#)